

LAPORAN PRAKTIKUM STRUKTUR DATA DAN ALGORITMA
DASAR PEMOGRAMAN JAVASCRIPT



DI SUSUN OLEH :

NAMA	: FARA MUTIA
NIM	: 2025573010034
PRODI	: TEKNIK INFORMATIKA
DOSEN PENGAJAR	: M. REZA ZULMAN, S.ST.,M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
LHOKSEUMAWE

2026

LEMBARAN PENGESAHAN

MATA KULIAH : Praktikum Struktur Data dan Algoritma
JUDUL PRAKTIKUM : Laporan Praktikum Dasar Pemograman JavaScript
HARI PRAKTIKUM : Kamis
TANGGAL : 22 September 2025
PENYERAHAN
NAMA : Fara Mutia
NIM : 2025573010034
PRODI/KELAS : Teknik Informatika/I D
NILAI :

Mahasiswa

Bukit Rata, 8 April 2026
Dosen Pembimbing

Fara Mutia

M. REZA ZULMAN
S.ST.,M.Sc

NIM. 2025573010034

NIP. 19920501 2022031005

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari tugas dalam mata kuliah Praktikum Struktur Data dan Algoritma yang bertujuan untuk memberikan pemahaman tentang sintax JavaScript mulai dari mengetahui variable, tipe data, operator, kondisional sampai perulangan.

Saya menyadari bahwa laporan ini tidak akan terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing, teman-teman, serta pihak-pihak yang telah memberikan bantuan baik secara langsung maupun tidak langsung dalam penyelesaian laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi semua pihak yang membacanya. Kritik dan saran yang konstruktif sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

Bukit Rata, 6 April 2026
Penulis

Fara Mutia
NIM. 2025573010034

DAFTAR ISI

LEMBARAN PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Rumusan Masalah.....	1
BAB II LANDASAN TEORI	2
2.1 Pemograman JavaScript.....	2
2.1.1 Pengertian JavaScript.....	2
2.1.2 Struktur Sintaksis dan Prinsip Penulisan Kode	2
2.1.3 Dinamika Variabel	2
2.1.4 Tipe Data.....	3
2.1.5 Sistem Operator: Logika, Aritmatika, dan Komparasi	4
2.1.6 Pernyataan Bersyarat (Conditionals)	5
2.1.7 Mekanisme Perulangan (Loops).....	5
BAB III METOLOGI PERCOBAAN	6
3.1 Mempersiapkan Project Praktikum	6
3.2 Mengerjakan Contoh, Latihan dan Tugas	6
3.3 Sumbit ke GitHub	15
BAB IV PEMBAHASAN DAN ANALISIS	18
4.1 Pembahasan.....	18
4.2 Analisis.....	18
BAB V PENUTUP	19
5.1 Kesimpulan	19
BAB VI DAFTAR PUSAKA.....	20

BAB I PENDAHULUAN

1.1 Latar Belakang

Praktikum ini dilaksanakan untuk memberikan pemahaman mendalam mengenai fondasi pemrograman JavaScript, khususnya pada penggunaan variabel, tipe data, dan operator aritmatika. Selain penguasaan sintaks dasar, aspek krusial yang dipelajari adalah struktur kendali aliran program (*control flow*) melalui penggunaan kondisional if-else dan perulangan for loop. Kemampuan untuk mengatur alur logika ini sangat penting agar program dapat mengambil keputusan secara dinamis berdasarkan data yang diterima.

Selain aspek teknis pengkodean, praktikum ini juga mengintegrasikan penggunaan sistem pengontrol versi (*version control system*) menggunakan Git dan GitHub. Hal ini dilakukan untuk mengenalkan alur kerja standar industri, di mana setiap perubahan kode harus didokumentasikan, dikelola identifikasinya, dan disimpan secara terorganisir di dalam repositori. Melalui kombinasi antara pemahaman logika pemrograman dan penggunaan alat bantu pengembang, praktikum ini bertujuan untuk membentuk landasan berpikir yang sistematis dan profesional dalam menghadapi kompleksitas pengembangan perangkat lunak di masa depan.

1.2 Tujuan Praktikum

1. Mengetahui dasar pemrograman JavaScript
2. Dapat membuat program sederhana dengan JavaScript
3. Mengetahui cara mengepush folder dari Git ke GitHub

1.3 Rumusan Masalah

1. Bagaimana cara push folder ke repository GitHub?
2. Apa itu mekanisme hoisting dan temporal dead zone?

BAB II LANDASAN TEORI

2.1 Pemrograman JavaScript

2.1.1 Pengertian JavaScript

JavaScript adalah bahasa pemrograman yang digunakan untuk membuat situs web menjadi lebih dinamis dan interaktif. Jika HTML adalah kerangka dan CSS adalah tampilannya, maka JavaScript adalah bagian yang memberikan logika dan interaksi. Bahasa ini dapat berjalan di sisi klien (melalui peramban seperti Google Chrome) maupun di sisi server (menggunakan Node.js).

JavaScript adalah bahasa multi-paradigma yang sangat fleksibel karena mendukung pendekatan prosedural, fungsional, dan berorientasi objek secara bersamaan. Kekuatannya terletak pada konsep fungsi sebagai objek kelas pertama yang mendukung pola fungsional, serta sistem pewarisan berbasis prototipe yang unik untuk kebutuhan pemrograman berorientasi objek. Dengan karakteristik ini, pengembang bebas memilih atau menggabungkan berbagai teknik sesuai dengan kompleksitas proyek yang dikerjakan.

2.1.2 Struktur Sintaksis dan Prinsip Penulisan Kode

Pemahaman mendalam terhadap aturan dasar merupakan kunci keberhasilan dalam mempelajari JavaScript. Sebagai bahasa yang *case-sensitive*, JavaScript membedakan variabel berdasarkan huruf besar dan kecil, serta memberikan fleksibilitas penulisan kode melalui tag `<script>` atau file eksternal (.js) yang lebih profesional. Pemisahan file eksternal sangat disarankan karena memudahkan pemeliharaan logika pemrograman secara terpisah dari struktur HTML.

Di sisi lain, meskipun mesin JavaScript cenderung mengabaikan *whitespace* dan menyisipkan titik koma secara otomatis, penggunaan semicolon yang konsisten tetap menjadi praktik terbaik untuk mencegah *bug*. Selain itu, penggunaan komentar sangat penting sebagai alat dokumentasi yang mempermudah kolaborasi antar pengembang.

2.1.3 Dinamika Variabel

Variabel dalam JavaScript adalah nama simbolis yang mewakili suatu nilai yang disimpan di dalam memori komputer. Sebelum era ES6, pengembang hanya memiliki satu kata kunci untuk mendeklarasikan variabel, yaitu `var`. Namun, keterbatasan `var`

dalam hal cakupan (scope) dan keamanan data mendorong diperkenalkannya `let` dan `const` sebagai standar baru dalam pengembangan modern.

Memahami perbedaan antara `var`, `let`, dan `const` adalah fondasi penting untuk menghindari kesalahan logika yang umum terjadi. Perbedaan utama antara `var`, `let`, dan `const` terletak pada cakupannya. `var` menggunakan *function scope*, sehingga variabel tetap dapat diakses di luar blok (seperti `if` atau `for`) selama masih dalam fungsi yang sama. Sebaliknya, `let` dan `const` menggunakan *block scope*, yang membatasi akses variabel hanya di dalam kurung kurawal `{}` tempat ia dideklarasikan untuk mencegah kesalahan logika kode.

2.1.4 Tipe Data

JavaScript adalah bahasa yang diketik secara dinamis (dynamically typed), di mana tipe data dari sebuah variabel ditentukan secara otomatis pada saat runtime berdasarkan nilai yang diberikan. Tipe data di JavaScript dibagi menjadi dua kategori besar: tipe data primitif dan tipe data objek.

a. Tipe Data Primitif

Tipe data primitif adalah tipe data dasar yang tidak dapat diubah (immutable).

1. **String:** Mewakili data tekstual, ditulis menggunakan tanda kutip tunggal (`'`), ganda (`"`), atau backtick (```) untuk template literals. Template literals memungkinkan interpolasi variabel dengan sintaks `${variable}`.
2. **Number:** Mewakili angka, baik bilangan bulat (integer) maupun angka desimal (float). JavaScript tidak memisahkan tipe data integer dan float secara eksplisit dalam kategori yang berbeda, semuanya berada di bawah tipe `Number`.
3. **Boolean:** Hanya memiliki dua nilai, yaitu `true` (benar) atau `false` (salah). Tipe data ini sangat penting dalam logika percabangan dan kontrol alur.
4. **Null:** Mewakili ketiadaan nilai yang disengaja. Pengembang menggunakan `null` untuk mengosongkan nilai variabel secara eksplisit.
5. **Undefined:** Mewakili variabel yang telah dideklarasikan tetapi belum diberi nilai. Ini adalah nilai default yang diberikan oleh mesin JavaScript.

b. Tipe Data Objek dan Array

Objek adalah struktur data kompleks yang menyimpan koleksi data dalam bentuk pasangan kunci dan nilai (key-value pairs). Objek memungkinkan pengembang untuk merepresentasikan entitas dunia nyata di dalam kode, di mana kunci mewakili properti

dan nilai mewakili data atau fungsi (metode). Array adalah jenis khusus dari objek yang digunakan untuk menyimpan daftar nilai yang terurut berdasarkan indeks numerik, dimulai dari nol.

2.1.5 Sistem Operator: Logika, Aritmatika, dan Komparasi

Operator adalah simbol yang digunakan untuk memproses data dan menghasilkan nilai baru. JavaScript memiliki set operator yang kaya untuk menangani berbagai kebutuhan komputasi.

a. Operator Aritmatika dan Penugasan

Operator aritmatika digunakan untuk melakukan perhitungan matematika standar. Selain penjumlahan (+), pengurangan (-), perkalian (*), dan pembagian (/), JavaScript juga menyediakan operator modulus (%) untuk mendapatkan sisa hasil bagi, serta operator pangkat (**). Operator penugasan (assignment) seperti = digunakan untuk memberikan nilai pada variabel, sementara operator gabungan seperti +=, -=, *=, dan /= memungkinkan pembaruan nilai variabel secara efisien.

b. Operator Perbandingan dan Kesamaan Ketat

Efektivitas logika pemrograman sangat bergantung pada penggunaan operator perbandingan yang tepat. JavaScript menyediakan operator perbandingan standar seperti lebih besar (>), lebih kecil (<), serta lebih besar atau sama dengan (>=) dan lebih kecil atau sama dengan (<=).

Sangat penting untuk membedakan antara operator kesamaan longgar (==) dan kesamaan ketat (===). Operator == melakukan konversi tipe data otomatis (type coercion) sebelum membandingkan nilainya, sehingga "5" == 5 akan menghasilkan true. Namun, operator === membandingkan nilai sekaligus tipe datanya, sehingga "5" === 5 akan menghasilkan false. Praktik pengembangan modern sangat menganjurkan penggunaan === untuk menghindari bug yang disebabkan oleh konversi tipe data yang tidak terduga.

c. Operator Logika dan Evaluasi Hubung Singkat

Operator logika digunakan untuk menggabungkan beberapa ekspresi boolean. Operator AND (&&) akan mengembalikan true hanya jika kedua operand bernilai benar, sedangkan operator OR (||) mengembalikan true jika salah satu operand bernilai benar. Operator NOT (!) berfungsi untuk membalikkan nilai boolean.

2.1.6 Pernyataan Bersyarat (Conditionals)

Pernyataan if, else if, dan else adalah cara paling umum untuk menjalankan blok kode secara kondisional. Jika sebuah kondisi bernilai true, maka blok di dalam kurung kurawal akan dijalankan; jika tidak, mesin akan memeriksa kondisi berikutnya atau menjalankan blok else.

Untuk kasus yang melibatkan banyak kemungkinan nilai dari satu ekspresi, pernyataan switch menyediakan struktur yang lebih rapi dan efisien daripada rangkaian if-else yang panjang. Setiap case dalam switch harus diakhiri dengan break untuk mencegah "fall-through", di mana kode pada case berikutnya dijalankan tanpa pengecekan. Selain itu, terdapat operator ternary (? :) yang merupakan bentuk singkat dari if-else untuk penugasan nilai tunggal berdasarkan kondisi.

2.1.7 Mekanisme Perulangan (Loops)

Perulangan memungkinkan eksekusi blok kode secara berulang selama kondisi tertentu terpenuhi.

- a. For Loop: Digunakan ketika jumlah iterasi sudah diketahui, terdiri dari inisialisasi, kondisi, dan ekspresi pembaruan.
- b. While Loop: Mengeksekusi blok kode selama kondisi tetap bernilai true. Kondisi diperiksa sebelum blok dijalankan.
- c. Do-While Loop: Menjamin bahwa blok kode akan dijalankan setidaknya satu kali, karena kondisi diperiksa setelah blok dieksekusi.
- d. For-Of dan For-In: Digunakan untuk iterasi pada struktur data iterable seperti Array atau properti pada Objek.

Penggunaan kata kunci break dapat menghentikan loop sepenuhnya, sementara continue akan melewati iterasi saat ini dan langsung menuju iterasi berikutnya.

BAB III METOLOGI PERCOBAAN

3.1 Mempersiapkan Project Praktikum

1. Buka App Git Bash. Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone. Caranya ketik **cd nama-repository-kamu**.

```
newuser@DESKTOP-JP3BRBO MINGW64 ~  
$ cd '/e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma'
```

2. Buat folder baru dengan nama praktikum-2 untuk menyimpan semua file pada pertemuan ini.

```
newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)  
$ mkdir praktikum-2  
  
newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)  
$ cd praktikum-2
```

3. Inisialisasi npm project. Perintah ini akan membuat file package.json secara otomatis dengan cara ketik **npm init -y**. Lalu verifikasi bahwa file package.json sudah berhasil dibuat dengan ketik **ls**.

```
newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma/praktikum-2 (main)  
$ npm init -y  
Wrote to E:\Praktikum Struktur Data\202573010034_strukturdatadanalgoritma\praktikum-2\package.json:  
  
{  
  "name": "praktikum-2",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}  
  
newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma/praktikum-2 (main)  
$ ls  
package.json
```

3.2 Mengerjakan Contoh, Latihan dan Tugas

3.2.1 Variabel

1. Membuat file baru dengan nama 01-variabel.js di dalam folder praktikum-2/

```
newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma/praktikum-2 (main)  
$ touch 01-variabel.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```
1  //---Deklarasi dengan let (nilai bisa diubah)---
2  let nama = 'Fara Mutia';
3  let umur = 18;
4  let kota = 'Aceh Timur';
5
6  console.log('=== Data Mahasiswa ===');
7  console.log('Nama   : ', nama);
8  console.log('Umur   : ', umur);
9  console.log('Kota   : ', kota);
10
11 //---Mengubah nilai let---
12 umur = 19;
13 console.log('Ulang tahun ! Umur Sekarang : ', umur);
14
15 //--- Deklarasi dengan const (nilai TIDAK bisa diubah)---
16 const jurusan = 'Teknik Informatika';
17 const tahunMasuk = 2025;
18
19 console.log('Jurusan      : ', jurusan);
20 console.log('Tahun Masuk : ', tahunMasuk);
21
```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 01-variabel.js** di terminal.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS E:\Praktikum Struktur Data\202573010034_strukturdatadanalgoritma\praktikum-2> node 01-variabel.js
=== Data Mahasiswa ===
Nama   : Fara Mutia
Umur   : 18
Kota   : Aceh Timur
Ulang tahun ! Umur Sekarang : 19
Jurusan      : Teknik Informatika
Tahun Masuk : 2025
PS E:\Praktikum Struktur Data\202573010034_strukturdatadanalgoritma\praktikum-2> |
```

3.2.2 Tipe Data

1. Buat file baru bernama 02-tipe-data.js di dalam folder praktikum-2/. Caranya sama seperti membuat file pertama yaitu dengan **touch 02-tipe-data.js**.
2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```

1 // --- 1. STRING (teks) ---
2 let namaMahasiswa = 'Azhari Fauzi';
3 let prodi = "Teknik Informatika";
4
5 let pengenalan = `Halo! Nama saya ${namaMahasiswa} dari ${prodi}.`;
6 console.log(pengenalan);
7 console.log('Panjang nama:', namaMahasiswa.length);
8
9 // --- 2. NUMBER (angka) ---
10 let nilaiUjian = 87;
11 let ipk = 3.75;
12 let suhuKulkas = -5;
13
14 console.log('Nilai Ujian:', nilaiUjian);
15 console.log('IPK:', ipk);
16 console.log('Suhu Kulkas:', suhuKulkas);
17
18 // --- 3. BOOLEAN (true/false) ---
19 let sudahLogin = true;
20 let sudahLulus = false;
21
22 console.log('Sudah login:', sudahLogin);
23 console.log('Sudah lulus:', sudahLulus);
24
25 // --- 4. NULL (nilai kosong yang disengaja) ---
26 let fotoProfil = null;
27 console.log('Foto profil:', fotoProfil);
28
29 // --- 5. UNDEFINED (belum diberi nilai) ---
30 let nomorTelepon;
31 console.log('No. Telepon:', nomorTelepon);
32
33 // --- Mengecek tipe data dengan typeof ---
34 console.log('--- Tipe Data ---');
35 console.log('namaMahasiswa:', typeof namaMahasiswa); // string
36 console.log('nilaiUjian:', typeof nilaiUjian); // number
37 console.log('sudahLogin:', typeof sudahLogin); // boolean
38 console.log('fotoProfil:', typeof fotoProfil); // object <- keanehan JS!
39 console.log('nomorTelepon:', typeof nomorTelepon); // undefined

```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketika **node 02-tipe-data.js** di terminal.

```

PS E:\Praktikum Struktur Data\202573010034_strukturdatadanalgoritma\praktikum-2> node .\02-tipe-data.js
Halo! Nama saya Azhari Fauzi dari Teknik Informatika.
Panjang nama: 12
Nilai Ujian : 87
IPK : 3.75
Suhu Kulkas : -5
Sudah login : true
Sudah lulus : false
Foto profil: null
No. Telepon: undefined
No. Telepon: undefined
--- Tipe Data ---
namaMahasiswa : string
nilaiUjian : number
sudahLogin : boolean
fotoProfil : object
nomorTelepon : undefined

```

3.2.3 Operator

1. Buat file baru bernama 03-operator.js di dalam folder praktikum-2/. Caranya sama seperti membuat file pertama yaitu dengan **touch 03-operator.js**.
2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```
1  let a = 17;
2  let b = 5;
3  console.log('=== Aritmatika ===');
4  console.log(`${a} + ${b} = ${a + b}`);
5  console.log(`${a} - ${b} = ${a - b}`);
6  console.log(`${a} * ${b} = ${a * b}`);
7  console.log(`${a} / ${b} = ${a / b}`);
8  console.log(`${a} % ${b} = ${a % b}`);
9  console.log(`${a} ** ${b} = ${a ** b}`);
10
11 console.log('=== Perbandingan ===');
12 console.log('10 > 5 :', 10 > 5);
13 console.log('10 < 5 :', 10 < 5);
14 console.log('10 >= 10 :', 10 >= 10);
15 console.log('10 <= 9 :', 10 <= 9);
16
17 console.log('--- Perbedaan == dan === ---');
18 console.log('5 == "5" :', 5 == '5');
19 console.log('5 === "5" :', 5 === '5');
20 console.log('5 !== 3 :', 5 !== 3);
21
22 console.log('\n=== Logika ===');
23 let sudahMakan = true;
24 let punyaUang = false;
25
26 console.log('Makan && Uang :', sudahMakan && punyaUang);
27 console.log('Makan || Uang :', sudahMakan || punyaUang);
28
29 console.log('!sudahMakan :', !sudahMakan);
30 console.log('!punyaUang :', !punyaUang);
```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 03-operator.js** di terminal.

```
PS E:\Praktikum Struktur Data\202573010034_strukturdatanalgoritma\praktikum-2> node .\03-operator.js
=== Aritmatika ===
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3.4
17 % 5 = 2
17 ** 5 = 1419857
=== Perbandingan ===
10 > 5 : true
10 < 5 : false
10 >= 10 : true
10 <= 9 : false
--- Perbedaan == dan === ---
5 == "5" : true
5 === "5" : false
5 !== 3 : true

=== Logika ===
Makan && Uang : false
Makan || Uang : true
!sudahMakan : false
!punyaUang : true
```

4. Lalu kerjakan Latihan dalam file yang sama (di bawah baris code contoh).

```
33 //latihan
34 const panjang = 12;
35 const lebar = 8;
36
37 const luas = panjang * lebar;
38 const keliling = 2 * (panjang + lebar);
39
40 console.log(`\nLuas persegi panjang: ${luas}`);
41 console.log(`Keliling persegi panjang: ${keliling}`);
42
43 const apakahLebihBesarDari100 = luas > 100;
44 console.log(`Apakah luas lebih besar dari 100? ${apakahLebihBesarDari100}`);
45
```

5. Hasil setelah dijalankan,

```
Luas persegi panjang: 96
Keliling persegi panjang: 40
Apakah luas lebih besar dari 100? false
```

3.2.4 Kondisional

1. Buat file baru bernama 04-kondisional.js di dalam folder praktikum-2/. Caranya sama seperti membuat file pertama yaitu dengan **touch 04-kondisional.js**.
2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```
1 let nilaiUjian = 78;
2
3 console.log('=== Konversi Grade ===');
4 console.log('Nilai:', nilaiUjian);
5
6 if (nilaiUjian >= 90) {
7     console.log('Grade: A (Sangat Baik)');
8 } else if (nilaiUjian >= 80) {
9     console.log('Grade: B (Baik)');
10 } else if (nilaiUjian >= 70) {
11     console.log('Grade: C (Cukup)');
12 } else if (nilaiUjian >= 60) {
13     console.log('Grade: D (Kurang)');
14 } else {
15     console.log('Grade: E (Tidak Lulus)');
16 }
```

```

18 let namaHari = 'Rabu';
19 console.log('\n=== Cek Jenis Hari ===');
20 switch (namaHari) {
21     case 'Senin':
22     case 'Selasa':
23     case 'Rabu':
24     case 'Kamis':
25     case 'Jumat':
26         console.log(`${namaHari} adalah hari kerja.`);
27         break;
28     case 'Sabtu':
29     case 'Minggu':
30         console.log(`${namaHari} adalah akhir pekan.`);
31         break;
32     default:
33         console.log('Nama hari tidak dikenali.');
```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 03-operator.js** di terminal.

```

=== Konversi Grade ===
Nilai: 78
Grade: C (Cukup)

=== Cek Jenis Hari ===
Rabu adalah hari kerja.

=== Status Kelulusan ===
Nilai 65: LULUS
```

4. Lalu kerjakan Latihan dalam file yang sama (di bawah baris code contoh).

```

41 //latihan
42 const bulan = 7;
43 console.log('\nBulan :', bulan);
44 if(bulan == 12 || bulan == 1 || bulan == 2){
45     console.log('musim dingin');
46 }
47 else if(bulan == 3 || bulan == 4 || bulan == 5){
48     console.log('Musim semi');
49 }
50 else if(bulan == 6 || bulan == 7 || bulan == 8){
51     console.log('Musim panas');
52 }
53 else if(bulan == 9 || bulan == 10 || bulan == 11){
54     console.log('Musim gugur');
55 }
56 const adaAwan = true;
57 const adaAngin = false;
58
59 console.log(`Apakah cuaca mendung sekaligus berangin? ${adaAwan && adaAngin}`);
60 console.log(`Apakah ada awan atau angin? ${adaAwan || adaAngin}`);
61 console.log(`Apakah langit cerah (tidak ada awan)? ${!adaAwan}`);
```

5. Hasil setelah dijalankan.

```
Bulan : 7
Musim panas
Apakah cuaca mendung sekaligus berangin? false
Apakah ada awan atau angin? true
Apakah langit cerah (tidak ada awan)? false
```

3.2.5 Perulangan

1. Buat file baru bernama 05-perulangan.js di dalam folder praktikum-2/. Caranya sama seperti membuat file pertama yaitu dengan **touch 02-tipe-data.js**.
2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```
1  console.log('=== For Loop: Hitung 1 sampai 5 ===');
2  for (let i = 1; i <= 5; i++) {
3    |  console.log(`Iterasi ke-${i}`);
4  }
5
6  console.log('\n=== While Loop: Countdown ===');
7  let hitung = 5;
8  while (hitung > 0) {
9    |  console.log(`Hitung mundur: ${hitung}`);
10   |  hitung--;
11  }
12  console.log('Selesai!');
13
14  console.log('\n=== Bilangan Genap antara 1 - 20 ===');
15  for (let i = 1; i <= 20; i++) {
16    |  if (i % 2 === 0) {
17    |  |  console.log(i + ' ');
18    |  }
19  }
20  console.log('');
21
22  console.log('\n=== Break dan Continue ===');
23  for (let i = 1; i <= 10; i++) {
24    |  if (i === 4) {
25    |  |  console.log(`Melewati angka ${i} (continue)`);
26    |  |  continue;
27    |  }
28    |  if (i === 8) {
29    |  |  console.log(`Berhenti di angka ${i} (break)`);
30    |  |  break;
31    |  }
32    |  console.log(`Angka: ${i}`);
33  }
```


3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 03-operator.js** di terminal.

```

=== For Loop: Hitung 1 sampai 5 ===
Iterasi ke-1
Iterasi ke-2
Iterasi ke-3
Iterasi ke-4
Iterasi ke-5
=== While Loop: Countdown ===
Hitung mundur: 5
Hitung mundur: 4
Hitung mundur: 3
Hitung mundur: 2
Hitung mundur: 1
Selesai!

=== Bilangan Genap antara 1 - 20 ===
2
4
6
8
10
12
14
16
18
20
=== Break dan Continue ===
Angka: 1
Angka: 2
Angka: 3
Melewati angka 4 (continue)
Angka: 5
Angka: 6
Angka: 7
Berhenti di angka 8 (break)

```

4. Lalu kerjakan Latihan dalam file yang sama (di bawah baris code contoh).

```

36 //latihan
37 for (let i = 1; i <= 5; i++) {
38   let bintang = "";
39   for (let j = 1; j <= i; j++) {
40     bintang += "*";
41   }
42   console.log(bintang);
43 }
44
45 for (let i = 2; i <= 50; i++) {
46   let Prima = true;
47
48   for (let j = 2; j < i; j++) {
49     if (i % j === 0) {
50       Prima = false;
51       break;
52     }
53   }
54   if (Prima) {
55     console.log(i);
56   }
57 }

```

Program

```

===segitiga bintang===
*
**
***
****
*****

===bilangan prima dari 2-50===
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47

```

Hasil

3.2.6 Tugas Praktikum

1. Buat folder tugas/

```

newuser@DESKTOP-JP3BR60 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadana algoritma/praktikum-2 (main)
$ mkdir tugas

```

2. Lalu tambahkan dua file tugas di dalam folder tugas/ tersebut yaitu tugas-1.js dan tugas-2.js.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma/praktikum-2/tugas (main)
$ touch tugas-1.js

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma/praktikum-2/tugas (main)
$ touch tugas-2.js
```

3. Program tugas 1 beserta hasil outputnya.

```
1  for(let i = 1; i <= 50; i++){
2
3      if(i % 15 === 0){
4          console.log('FizzBuzz');
5      }else if(i % 3 === 0){
6          console.log('Fizz');
7      }else if(i % 5 === 0){
8          console.log('Buzz');
9      }else
10         console.log(`${i}`);
11
12 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgoritma\praktikum-2> node .\tugas\tugas1.js
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
27
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
```

4. Program tugas 2 beserta hasil outputnya.

```

1  const bb = 50;
2  const tb = 1.72;
3
4  console.log('Berat :', bb);
5  console.log('Tinggi :', tb);
6
7  ibm = bb / (tb * tb);
8  console.log('IBM :', ibm.toFixed(2));
9
10 if (ibm < 18.5){
11   console.log('Kategori : Kurus(Underweight)')
12 }else if(ibm >= 18.5 && ibm <= 24.9){
13   console.log('Kategori : Normal(ideal)');
14 }else if(ibm >= 25.0 && ibm <= 29.9){
15   console.log('Kategori : Kelebihan Berat Badan(Overweight)');
16 }else
17   console.log('Kategori : Obesitas(Obese)');
18

```

```

Berat : 50
Tinggi : 1.72
IBM : 16.90
Kategori : Kurus(Underweight)

```

```

Berat : 68
Tinggi : 1.72
IBM : 22.99
Kategori : Normal(ideal)

```

```

Berat : 85
Tinggi : 1.72
IBM : 28.73
Kategori : Kelebihan Berat Badan(Overweight)

```

```

Berat : 90
Tinggi : 1.72
IBM : 30.42
Kategori : Obesitas(Obese)

```

3.3 Sumbit ke GitHub

Setelah semua tugas, latihan, dan contoh selesai, lakukan push ke *repository* GitHub.

1. Ketik `cd ..` di Git Bush agar tidak berada dalam `/praktikum-2` tetapi di root folder repository kita. Karena Git diinisialisasi di folder utama (root). Kalau kita di dalam folder `praktikum-2`, Git mungkin tidak akan melihat perubahan di folder lain.

```

newuser@DESKTOP-JP3BRB0 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-2 (main)
$ cd ..

newuser@DESKTOP-JP3BRB0 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma (main)
$ |

```

2. Ketik `git status` untuk cek status file, lalu ketik `git add praktikum -2/` untuk menambahkan

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    praktikum-2/

nothing added to commit but untracked files present (use "git add" to track)

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma (main)
$ git add praktikum-2/
warning: in the working copy of 'praktikum-2/package.json', LF will be replaced
by CRLF the next time Git touches it

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   praktikum-2/01-variabel.js
    new file:   praktikum-2/02-tipe-data.js
    new file:   praktikum-2/03-operator.js
    new file:   praktikum-2/04-kondisional.js
    new file:   praktikum-2/05-perulangan.js
    new file:   praktikum-2/package.json
    new file:   praktikum-2/tugas/tugas1.js
    new file:   praktikum-2/tugas/tugas2.js

```

3. Buat commit dengan pesan yang deskriptif. Seperti `git commit -m "Praktikum 2: Dasar JavaScript - variabel, tipe data, operator, kondisional, perulangan"` tetapi disini ada masalahnya karena Git belum tahu siapa kamu. Setiap kali kamu melakukan git commit, Git perlu mencatat nama dan email penulisnya supaya sejarah perubahan kodenya jelas.

```

ACER@DESKTOP-204TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanaalgoritma (main)
$ git commit -m "Praktikum 2 : Dasar JavaScript - variabel, tipe data, operator,
kondisional,perulangan"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

Fatal: unable to auto-detect email address (got 'ACER@DESKTOP-204TJ2F.(none)')

ACER@DESKTOP-204TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanaalgoritma (main)
$ git config --global user.email "anisahnabila81@gmail.com"

ACER@DESKTOP-204TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_

```

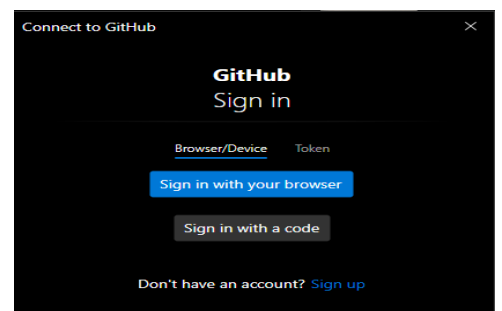
- Setelah memasukkan email dan nama buat lagi commit ulang

```
ACER@DESKTOP-204TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanaalgoritma (main)
$ git commit -m "praktikum 2 : Dasar Javascript - variabel, tipe data, operator,
kondisional, perulangan"
[main b40fb27] praktikum 2 : Dasar Javascript - variabel, tipe data, operator, k
ondisional, perulangan
8 files changed, 285 insertions(+)
create mode 100644 praktikum-2/01-variabel.js
create mode 100644 praktikum-2/02-tipe-data.js
create mode 100644 praktikum-2/03-operator.js
create mode 100644 praktikum-2/04-kondisional.js
create mode 100644 praktikum-2/05-perulangan.js
create mode 100644 praktikum-2/package.json
create mode 100644 praktikum-2/tugas/tugas-1.js
create mode 100644 praktikum-2/tugas/tugas-2.js
```

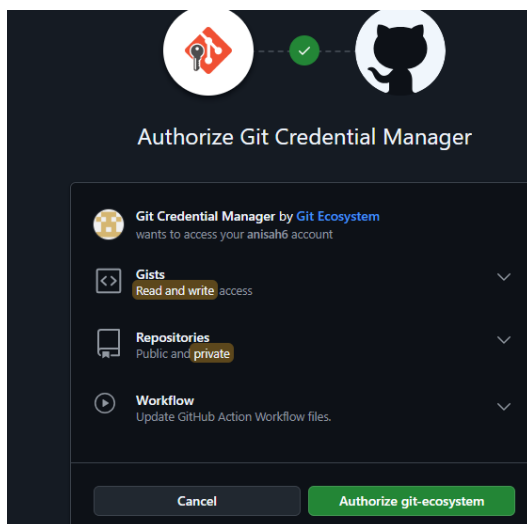
- Lalu lakukan push ke repository GitHub.

```
strukturdatadanaalgoritma (main)
$ git push origin main
```

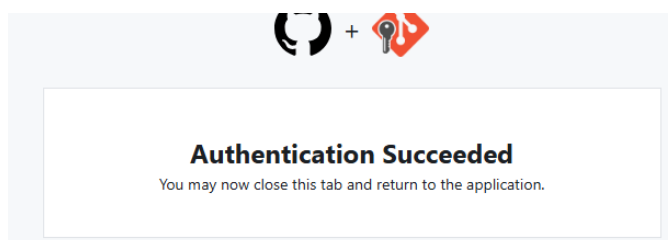
- Setelah di enter akan muncul gambar seperti di samping ini dan pilih *sign in with your browser*.
- Lalu masukan username dan password kalian dan tekan sign in



- Selanjutnya melakukan otorisasi (izin) agar laptop dapat terhubung langsung dengan akun GitHub. Langsung klik Authorize git-ecosystem.



- Selesai.



BAB IV PEMBAHASAN DAN ANALISIS

4.1 Pembahasan

Pada tugas-1 kenapa harus menaruh cek % 3 atau % 5 di bagian paling atas, karena logika pemrograman berjalan secara berurutan dari atas ke bawah, misal cek % 3 dulu. Saat program sampai ke angka 15, dia akan mengecek "Apakah 15 habis dibagi 3?". Jawabannya IYA. Program akan langsung mencetak "Fizz" dan mengabaikan kondisi di bawahnya. Padahal seharusnya 15 adalah "FizzBuzz".

Mekanisme Hoisting dan Temporal Dead Zone

Hoisting merupakan perilaku mesin JavaScript yang secara otomatis memindahkan deklarasi variabel dan fungsi ke bagian atas lingkungannya (scope) sebelum kode dieksekusi. Pada penggunaan `var`, hoisting memungkinkan variabel diakses sebelum baris deklarasinya, namun nilai yang dikembalikan adalah `undefined`. Hal ini sering menjadi sumber bug karena variabel sudah memiliki alokasi memori tetapi belum memiliki nilai yang valid. Sebaliknya, pada `let` dan `const`, variabel tetap di-hoist namun berada dalam Temporal Dead Zone (TDZ). Kondisi ini menyebabkan program memicu `ReferenceError` jika variabel diakses sebelum dideklarasikan, sehingga secara teknis memaksa pengembang untuk menulis kode yang lebih tertib, logis, dan aman dari kesalahan referensi.

Analisis Mendalam: Null vs Undefined

Meskipun secara fungsional keduanya mewakili ketiadaan nilai, terdapat perbedaan filosofis dan teknis yang mendalam antara `null` dan `undefined`. `Undefined` biasanya muncul secara otomatis sebagai akibat dari variabel yang tidak diinisialisasi, fungsi yang tidak memiliki nilai kembali, atau properti objek yang tidak ada. Sebaliknya, `null` adalah nilai penugasan yang menunjukkan bahwa suatu variabel secara sadar dikosongkan oleh pengembang.

4.2 Analisis

Berdasarkan hasil pengerjaan tugas, perulangan `for` digunakan untuk mengotomatisasi pencetakan angka dari 1 hingga 50 secara efisien. Dalam proses ini, ditemukan bahwa urutan logika pada struktur kondisional memegang peranan yang sangat vital. Penempatan kondisi angka yang habis dibagi 3 dan 5 (atau kelipatan 15) harus diprioritaskan di urutan paling atas sebelum pengecekan kondisi lainnya. Hal ini

dikarenakan logika pemrograman bekerja secara sekuensial dari atas ke bawah; jika kondisi yang lebih umum (seperti habis dibagi 3 saja) diletakkan di awal, maka angka seperti 15 akan langsung dianggap sebagai "Fizz" dan program akan mengabaikan kondisi "FizzBuzz". Penggunaan operator modulo (%) terbukti menjadi metode paling efektif untuk mendeteksi kelipatan angka. Selain itu, integrasi dengan Git menunjukkan bahwa konfigurasi identitas pengguna sangat diperlukan agar setiap perubahan dalam repositori dapat teratribusi dengan benar kepada pengembangnya.

BAB V PENUTUP

5.1 Kesimpulan

Dapat disimpulkan bahwa keberhasilan dalam membangun program "FizzBuzz" sangat bergantung pada ketelitian pengembang dalam menyusun hierarki logika, di mana kondisi yang paling spesifik harus didahulukan daripada kondisi yang bersifat umum. Praktikum ini juga memperkuat pemahaman bahwa pemrograman bukan sekadar menulis baris kode, melainkan tentang bagaimana memecahkan masalah komputasi melalui urutan perintah yang logis dan terstruktur. Penggunaan alat bantu seperti Git dan GitHub juga menjadi bagian yang tidak terpisahkan dalam siklus pengembangan perangkat lunak modern untuk menjaga integritas dan riwayat kode. Pada akhirnya, program yang dibuat telah berhasil berjalan sesuai instruksi dengan menghasilkan output yang konsisten melalui kombinasi perulangan dan percabangan yang tepat.

BAB VI DAFTAR PUSAKA

- Anamisa, D. R., & Mufarroha, F. A. (2022). *Dasar Pemrograman WEB Teori dan Implementasi: HTML, CSS, Javascript, Bootstrap, CodeIgniter*. Media Nusa Creative (MNC Publishing).
- Duckett, J. (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Indianapolis: John Wiley & Sons.
- Flanagan, D. (2020). *JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language*. Sebastopol: O'Reilly Media.
- GitHub Docs. (2026). *About Git*. Diakses dari <https://docs.github.com/en/get-started/using-git/about-git>
- MDN Web Docs. (2026). *Hoisting*. Mozilla Developer Network. Diakses dari <https://developer.mozilla.org/en-US/docs/Glossary/Hoisting>
- MDN Web Docs. (2026). *Expressions and operators: Arithmetic operators*. Mozilla Developer Network. Diakses dari <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators>